

✓ A Precisão do Pixel: Segmentação Semântica e de Instâncias

✓ Além das Caixas Delimitadoras: Um Entendimento Granular da Cena

A Próxima Fronteira da Percepção

Nossa jornada até aqui nos levou por uma escada de abstração crescente. Começamos com pixels (Aula 1), aprendemos a descrevê-los com características (Aula 2), a classificar imagens inteiras com base nessas características (Aula 3 e 4) e, mais recentemente, a localizar múltiplos objetos usando caixas delimitadoras (Aula 5). A detecção de objetos com **YOLO** nos deu uma resposta poderosa para as perguntas "**o que**" e "**onde**", mas a localização ainda é uma aproximação — **um retângulo**.

Para muitas aplicações críticas, uma caixa não é suficiente. **Um carro autônomo não precisa apenas saber que há um pedestre em uma caixa à sua frente; ele precisa saber o contorno exato do pedestre** para planejar uma trajetória segura. **Um software de análise de imagens médicas precisa delinear a fronteira exata de um tumor**, não apenas uma caixa ao seu redor.

É aqui que entramos na **Segmentação de Imagens**, a tarefa de classificar cada pixel de uma imagem. Em vez de uma única etiqueta por imagem ou uma caixa por objeto, **o objetivo é gerar uma máscara de segmentação**, uma imagem onde cada pixel recebe um rótulo correspondente à classe do objeto ao qual pertence. Isso nos dá o entendimento mais detalhado possível de uma cena.

✓ O Espectro da Segmentação: Semântica vs. Instância

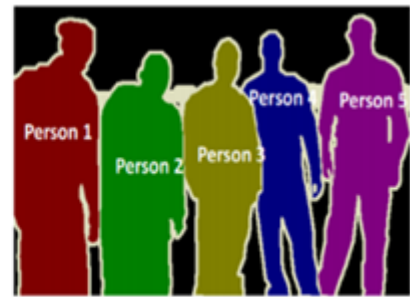
A tarefa de segmentar uma imagem se divide em duas categorias principais, que respondem a perguntas sutilmente diferentes, mas com implicações profundas.



Detecção de objetos



Segmentação semântica



Segmentação de instâncias

Segmentação Semântica: O Que é "Coisa" e o Que é "Cenário"?

A **segmentação semântica** classifica cada pixel da imagem em uma categoria. Ela **não diferencia instâncias individuais da mesma classe**. Pense nela como colorir um desenho: todos os pixels que pertencem a um carro serão pintados de "azul-carro", todos os pixels de estrada serão "cinza-estrada" e todos os pixels de céu serão "ciano-céu". **Se houver três carros estacionados juntos, a segmentação semântica os tratará como uma única mancha "azul-carro"**.

- **Pergunta que responde:** "A que classe de objeto este pixel pertence?"
- **Foco:** Diferenciar entre "coisas" (objetos contáveis como carros, pessoas) e "cenário" (stuff, regiões amorfas como céu, grama, estrada).
- **Aplicação:** Essencial para veículos autônomos entenderem áreas navegáveis (estrada vs. calçada) ou para análise de imagens de satélite (floresta vs. água vs. área urbana).

✓ Segmentação de Instância: Quantos e Quais São?

A segmentação de instância vai um passo além. **Ela não apenas classifica cada pixel, mas também distingue cada instância de objeto**. No nosso exemplo dos três carros, a segmentação de instância geraria três máscaras distintas: "carro 1", "carro 2" e "carro 3". Ela combina a **detecção de objetos** (encontrar objetos individuais) com a **segmentação semântica** (delinear seus contornos).

- **Pergunta que responde:** "A que instância de objeto única este pixel pertence?"
- **Foco:** Contar e delinear objetos individuais.
- **Aplicação:** Vital para contar células em uma imagem de microscópio, rastrear múltiplos jogadores em um campo de futebol ou em sistemas de checkout autônomo em lojas, onde cada produto precisa ser identificado individualmente.

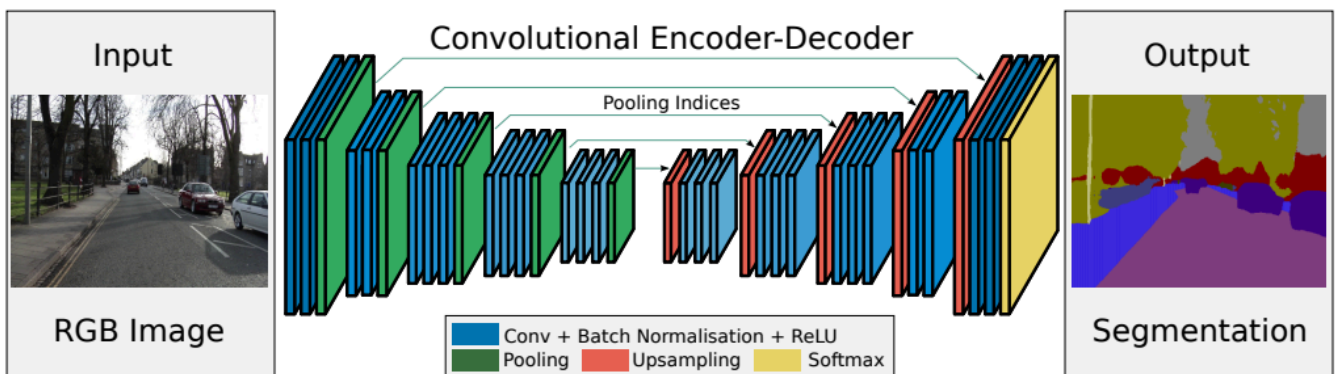
Característica	Segmentação Semântica	Segmentação de Instância
Objetivo	Classificar cada pixel em uma categoria.	Identificar e delinear cada instância de objeto.
Diferenciação	Não distingue objetos da mesma classe.	Distingue objetos da mesma classe.
Saída	Um único mapa de segmentação para a imagem.	Uma máscara de segmentação para cada objeto detectado
Analogia	Colorir por categoria (todos os gatos de azul).	Recortar cada gato individualmente.

Existe ainda a **segmentação panóptica**, que unifica as duas: atribui a cada pixel tanto um **rótulo de classe** (semântico) quanto um **ID de instância**, cobrindo tanto "coisas" quanto "cenário" em uma única saída coesa.

✓ Arquiteturas de Deep Learning para Segmentação

Para realizar a segmentação em nível de pixel, as arquiteturas de **CNN** precisam ser adaptadas. Uma CNN de classificação padrão, como a que construímos na Aula 4, reduz a resolução espacial da imagem até chegar a um único vetor de previsão. **Para a segmentação, precisamos de uma saída que tenha a mesma resolução da imagem de entrada.** A solução para isso é a arquitetura **Encoder-Decoder**.

✓ O Padrão Encoder-Decoder

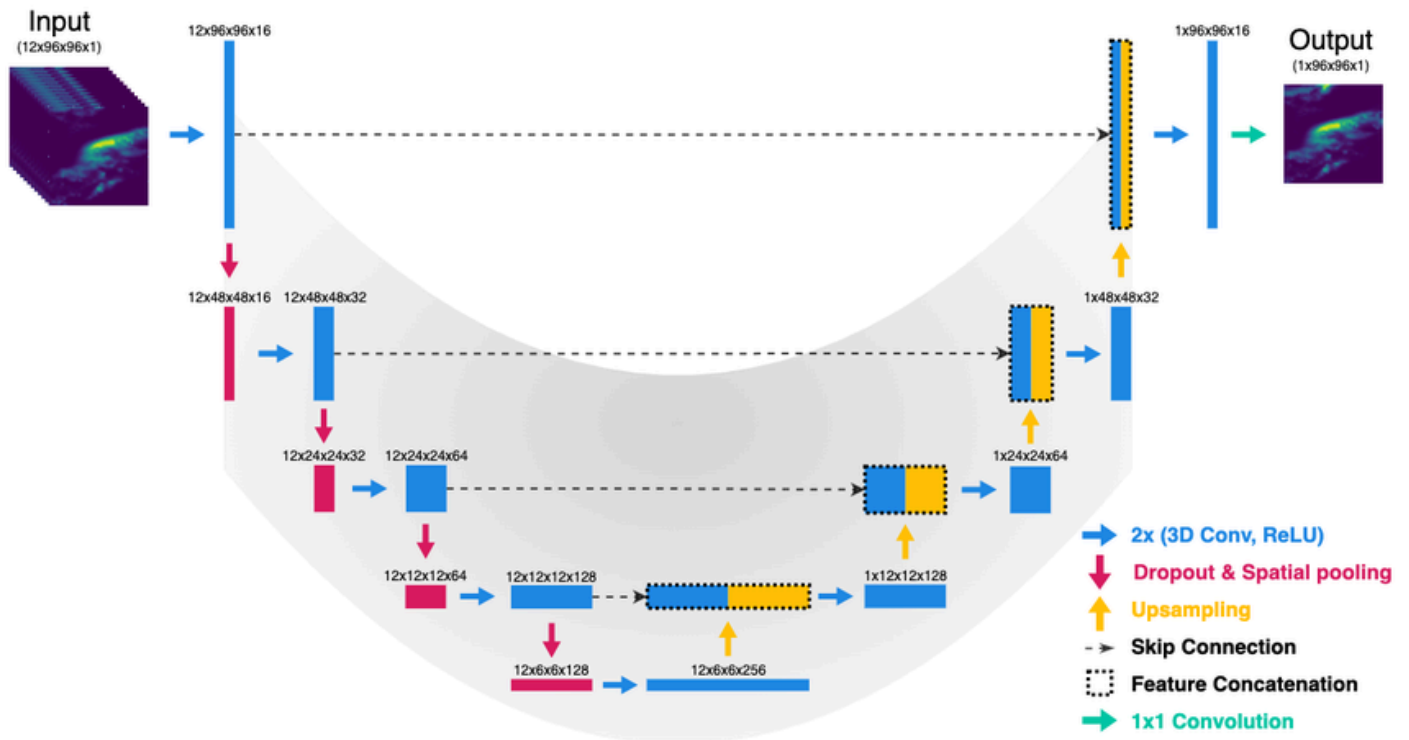


Essa arquitetura consiste em duas partes principais:

1. **O Encoder (Caminho de Contração):** Esta parte é muito semelhante a uma **CNN** de classificação. **É uma pilha de camadas convolucionais e de pooling que progressivamente reduzem a resolução espacial da imagem. O objetivo do encoder é aprender a extrair características semânticas e contextuais da imagem, descobrindo o que está presente na cena.**
2. **O Decoder (Caminho de Expansão):** O decoder pega a representação de características de baixa resolução do encoder e a expande, ou **faz upsampling**, de volta à resolução original da imagem. **Seu objetivo é usar as características aprendidas pelo encoder para localizar onde essas características estão, gerando o mapa de segmentação final.** O upsampling é geralmente realizado por uma camada especial chamada **Convolução Transposta** (às vezes chamada de "deconvolução").

✓ U-Net: A Importância das Skip Connections

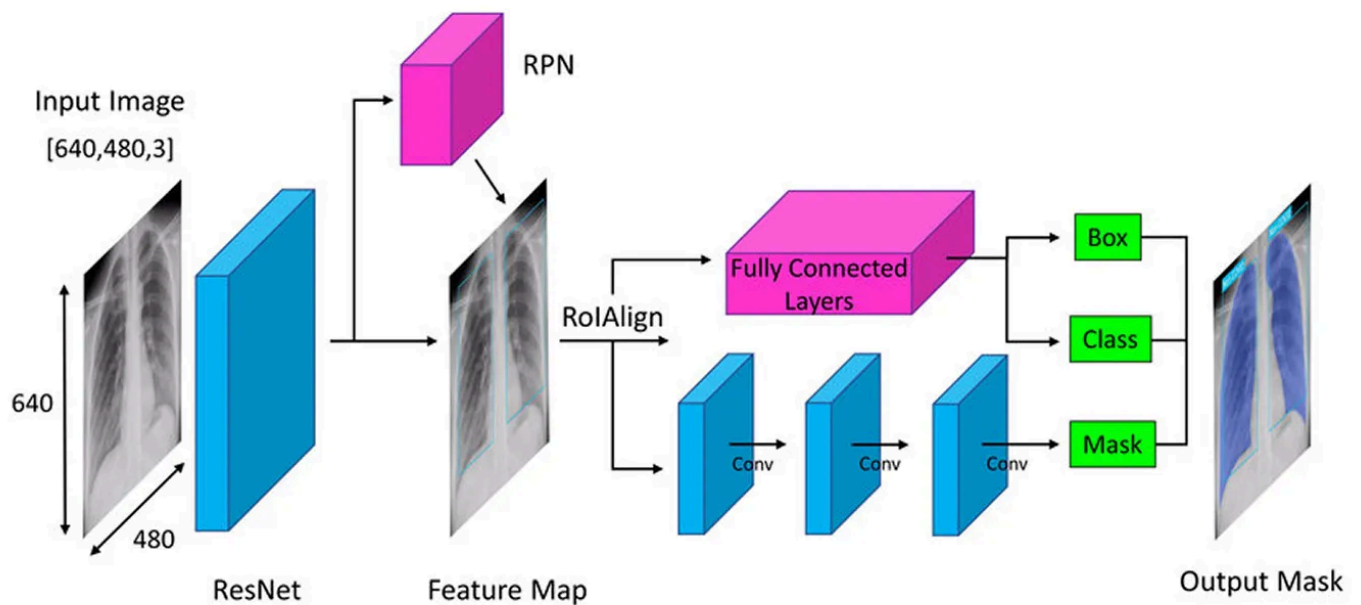
Uma das arquiteturas **encoder-decoder** mais famosas e influentes é a **U-Net**, nomeada por sua forma de "U". Sua grande inovação foi a introdução de **skip connections** (conexões de atalho).



- **O Problema:** Durante o processo de downsampling no encoder, informações espaciais de alta resolução (como bordas precisas) são perdidas. O decoder, trabalhando apenas com a representação de baixa resolução do final do encoder, tem dificuldade em reconstruir os contornos exatos dos objetos.
- **A Solução da U-Net:** As skip connections resolvem isso conectando diretamente as saídas das camadas do encoder às camadas correspondentes no decoder. Isso permite que o decoder acesse diretamente as características de alta resolução das etapas iniciais do encoder. O decoder pode, então, combinar a informação semântica de baixo nível (do caminho profundo) com a informação espacial de alto nível (das skip connections) para produzir máscaras de segmentação muito mais precisas.

✓ Mask R-CNN: A Abordagem "Detectar e Depois Segmentar"

Para a segmentação de instância, uma abordagem dominante é a da família **R-CNN**, que culminou na **Mask R-CNN**. A estratégia é de duas etapas: **primeiro detectar, depois segmentar**.



1. **Baseada na Faster R-CNN:** A Mask R-CNN começa exatamente como a **Faster R-CNN** (que discutimos conceitualmente na Aula 5). Ela usa uma **CNN de backbone para extrair características** e uma **Rede de Proposta de Região (RPN)** para gerar **caixas delimitadoras candidatas onde os objetos podem estar**.
2. **Três Tarefas em Paralelo:** Para cada **região de interesse (RoI)** proposta, a **Mask R-CNN** executa três tarefas em paralelo:
 1. **Classificação:** Determina a classe do objeto na caixa.
 2. **Regressão da Caixa:** Refina as coordenadas da caixa delimitadora para se ajustar melhor ao objeto.
 3. **Geração da Máscara:** Esta é a adição crucial. Uma pequena rede totalmente convolucional (FCN) é aplicada à RoI para gerar uma máscara de segmentação binária, pixel a pixel, para o objeto dentro daquela caixa.
3. **RoIAlign:** Uma inovação chave da Mask R-CNN é a camada RoIAlign. As abordagens anteriores (como a RoIPool da Fast R-CNN) usavam quantização (arredondamento) para mapear a RoI para o mapa de características, o que causava desalinhamentos que prejudicavam a precisão da máscara. **A RoIAlign usa interpolação bilinear para calcular os valores das características de forma precisa**, preservando o alinhamento espacial e permitindo a geração de máscaras muito mais detalhadas.

✓ Laboratório Prático: Segmentação de Instâncias com YOLOv8

Nesta seção, vamos aplicar um dos modelos mais modernos e populares para tarefas de visão computacional: o **YOLOv8**. Embora seja famoso pela detecção de objetos, ele **possui versões pré-treinadas especificamente para segmentação de instâncias**, que são extremamente fáceis de usar e oferecem resultados de ponta.

Preparação do Ambiente

A configuração é a mais simples possível. Precisamos apenas instalar a biblioteca

`ultralytics`, que contém o **YOLOv8**.

```
# Instala a biblioteca ultralytics que contém o YOLOv8
```

```
!pip install ultralytics
```

```
Requirement already satisfied: numpy<2.0.0,>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: opencv-python!=4.13.0.90, >=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Requirement already satisfied: nvidia-ml-py>=12.0.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics==8.0.110)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics_thop-2.0.20-py3-none-any.whl.metadata (14 kB)
```

```
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics==8.0.110)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics==8.0.110)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics==8.0.110)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics==8.0.110)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics==8.0.110)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics==8.0.110)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics==8.0.110)
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultralytics==8.0.110)
Requirement already satisfied: charset-normalizer<4, >=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics==8.0.110)
Requirement already satisfied: idna<4, >=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics==8.0.110)
Requirement already satisfied: urllib3<3, >=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics==8.0.110)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics==8.0.110)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics==8.0.110)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: cuda-bindings<13, >=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from torchvision>=0.9.0->ultralytics==8.0.110)
```


1.4/1.4 MB 39.7 MB/s eta 0:00:00
Downloading ultralytics_thop-2.0.20-py3-none-any.whl (28 kB)
Installing collected packages: ultralytics-thop, ultralytics
Successfully installed ultralytics-8.4.105 ultralytics-thop-2.0.20

✓ Implementação da Segmentação de Instâncias

O script a seguir irá baixar uma imagem de exemplo, carregar o modelo **YOLOv8 pré-treinado para segmentação** (`yolov8n-seg.pt`) - a versão "nano", que é a mais leve e rápida) e executará a inferência para gerar as máscaras de cada objeto detectado.

Importação das bibliotecas necessárias

```
import cv2
import numpy as np
from ultralytics import YOLO
from matplotlib import pyplot as plt
import requests # Para baixar a imagem de exemplo

import requests
import os
from google.colab import drive
```

Creating new Ultralytics Settings v0.0.6 file 
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/setting
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/di

✓ Preparação da imagem e do modelo

```
# Dicionário com os links das imagens do Google Drive e seus nomes
image_options = {
    "1": {"name": "rua_movimentada", "id": "1rocGB_adHySun2LcdclJdTmwASf7zD9q"},
    "2": {"name": "animal_natural", "id": "1hvJyhm5cBtQtCzqM1QI4nfb-LrMOS1P"},
    "3": {"name": "ambiente_interno", "id": "1Jt00zsUTS15w-WQRwsENgCHR1kjGLBin"}
}

def download_file_from_google_drive(file_id, destination):
    URL = "https://docs.google.com/uc?export=download"
    session = requests.Session()
    response = session.get(URL, params={'id': file_id}, stream=True)
    token = get_confirm_token(response)

    if token:
        params = {'id': file_id, 'confirm': token}
        response = session.get(URL, params=params, stream=True)

    save_response_content(response, destination)

def get_confirm_token(response):
    for key, value in response.cookies.items():
        if key.startswith('download_warning'):
            return value
    return None
```

```

def save_response_content(response, destination):
    CHUNK_SIZE = 32768
    with open(destination, "wb") as f:
        for chunk in response.iter_content(CHUNK_SIZE):
            if chunk:
                f.write(chunk)

# Permitir ao usuário escolher qual imagem baixar
print("Escolha uma imagem para baixar:")
for key, value in image_options.items():
    print(f"{key}: {value['name'].replace('_', ' ').capitalize()}")

choice = input("Digite o número da sua escolha (1, 2 ou 3): ")

selected_image = image_options.get(choice)

if selected_image:
    image_filename = f"{selected_image['name']}.jpg"
    file_id = selected_image['id']
    print(f"Baixando {image_filename}...")
    download_file_from_google_drive(file_id, image_filename)
    print("Download concluído.")
else:
    print("Escolha inválida. Usando a imagem de rua movimentada como padrão.")
    image_filename = f"{image_options['1']['name']}.jpg"
    file_id = image_options['1']['id']
    download_file_from_google_drive(file_id, image_filename)

# O cv2.imread será executado no próximo bloco que necessita da imagem já baixada.
# Removido cv2.imread daqui para evitar erro de imagem não encontrada antes do dowr

```

```

Escolha uma imagem para baixar:
1: Rua movimentada
2: Animal natural
3: Ambiente interno
Digite o número da sua escolha (1, 2 ou 3): 3
Baixando ambiente_interno.jpg...
Download concluído.

```

Carregar o modelo **YOLOv8** pré-treinado para **segmentação**.

A primeira vez que você executar isso, a biblioteca fará o download automático dos pesos do modelo. `yolo8n-seg.pt` é o modelo mais leve, ideal para experimentação rápida.

```
model = YOLO('yolo8n-seg.pt')
```

Carregar a imagem original com OpenCV para manipulação

```

import cv2
from google.colab.patches import cv2_imshow

image = cv2.imread(image_filename) # Usa image_filename ao invés de IMAGE_PATH

# Verifica se a imagem foi carregada corretamente antes de converter
if image is None:
    print(f"Erro: Não foi possível carregar a imagem em '{image_filename}'. Verifique

```


else:

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
cv2_imshow(image)
```



✓ Execução da inferência

```
print("Executando inferência de segmentação...")
# O método model() executa a predição na imagem.
# O resultado é uma lista de objetos 'Results'.
results = model(image_filename)
print("Inferência concluída.")
```

Executando inferência de segmentação...

image 1/1 /content/ambiente_interno.jpg: 448x640 3 chairs, 2 couches, 4 potted plants, 1
Speed: 2.2ms preprocess, 10.0ms inference, 2.3ms postprocess per image at shape (1, 3,
Inferência concluída.

✓ Processamento e visualização dos resultados

O objeto `results` contém todas as informações da primeira imagem. Vamos extrair as máscaras, caixas delimitadoras e classes.

```
result = results[0]
result
```

```

[[102, 104, 104],
 ...,
 [141, 136, 135],
 [141, 136, 135],
 [141, 136, 135]],

[[119, 121, 121],
 [114, 116, 116],
 [105, 107, 107],
 ...,
 [141, 136, 135],
 [141, 136, 135],
 [141, 136, 135]],

[[119, 121, 121],
 [117, 119, 119],
 [114, 116, 116],
 ...,
 [141, 136, 135],
 [141, 136, 135],
 [141, 136, 135]],

...,

[[182, 188, 183],
 [182, 188, 183],
 [184, 186, 186],
 ...,
 [121, 133, 137],
 [120, 132, 136],
 [120, 132, 136]],

[[182, 188, 183],
 [182, 188, 183],
 [184, 186, 186],
 ...,
 [121, 133, 137],
 [120, 132, 136],
 [120, 132, 136]],

[[182, 188, 183],
 [182, 188, 183],
 [184, 186, 186],
 ...,
 [121, 133, 137],
 [120, 132, 136],
 [120, 132, 136]]], dtype=uint8)
orig_shape: (452, 678)
path: '/content/ambiente_interno.jpg'
probs: None
save_dir: '/content/runs/segment/predict'
semantic_mask: None
speed: {'preprocess': 2.2476389999610547, 'inference': 10.033840000005512,
'postprocess': 2.300375000001351}

```

Criar uma cópia da imagem para desenhar a sobreposição

```
overlay = image_rgb.copy()
```

Gerar cores aleatórias para cada instância detectada

```
np.random.seed(42) # Para resultados reproduzíveis
colors = [np.random.randint(0, 255, size=3).tolist() for _ in range(len(result.masks))]
```

```
if result.masks is not None:
    # Iterar sobre cada objeto detectado (each mask)
    for i, mask in enumerate(result.masks.data):
        # Obter a máscara binária (0s e 1s)
        mask_data = mask.cpu().numpy().astype(np.uint8)

        # Redimensionar a máscara para o tamanho da imagem original
        mask_resized = cv2.resize(mask_data, (image.shape[1], image.shape[0]), interpo

        # Encontrar os pixels onde a máscara é 1 (ou seja, onde o objeto está)
        object_pixels = mask_resized == 1

        # Aplicar a cor da máscara na imagem de sobreposição
        overlay[object_pixels] = (overlay[object_pixels] * 0.4 + np.array(colors[i]) * 0.6)
```

✓ Exibir os resultados

A biblioteca **ultralytics** também tem uma função de plotagem muito conveniente que mostra caixas, máscaras e rótulos automaticamente. Vamos usá-la para uma visualização completa e rápida.

```
plotted_image_bgr = result.plot()
plotted_image_rgb = cv2.cvtColor(plotted_image_bgr, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(20, 7))

plt.subplot(1, 3, 1)
plt.title('Imagem Original')
plt.imshow(image_rgb)
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title('Máscaras Sobrepostas (Nossa Visualização)')
plt.imshow(overlay)
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title('Resultado Completo (YOLOv8 Plot)')
plt.imshow(plotted_image_rgb)
plt.axis('off')

plt.tight_layout()
plt.show()
```



Desafios Práticos

É hora de experimentar e ver a segmentação em ação.



Desafio 1: O Explorador de Genas



Tarefa: Execute o código do laboratório com pelo menos três imagens diferentes e variadas:

1. Uma imagem de uma rua movimentada (com vários carros, pessoas, etc.).
2. Uma imagem de um animal em um ambiente natural.
3. Uma imagem de um ambiente interno (uma sala de estar com um sofá, uma TV, etc.).

Análise: Observe os resultados. O modelo conseguiu identificar corretamente as diferentes classes? Como ele lida com objetos sobrepostos na segmentação semântica? Existem áreas onde a segmentação falha ou é imprecisa? Anote suas observações.

Ele conseguiu mesmo em objetos sobrepostos, diferenciar exatamente cada um dos objetos, na rua movimentada, até um relógio foi identificado, no ambiente fechado, nenhum sofá deixou de ser identificado

Desafio 2: O Detetive de Instâncias (Conceitual)

Tarefa: Encontre um projeto de código aberto que implemente a **Mask R-CNN** (o repositório da Matterport no GitHub é um excelente ponto de partida). Siga as instruções para configurar o ambiente e executar o notebook de demonstração (`demo.ipynb`) com uma imagem que contenha múltiplas instâncias do mesmo objeto (por exemplo, uma foto com vários balões,